

# CS541 Project 2: BRDF Lighting

## Synopsis

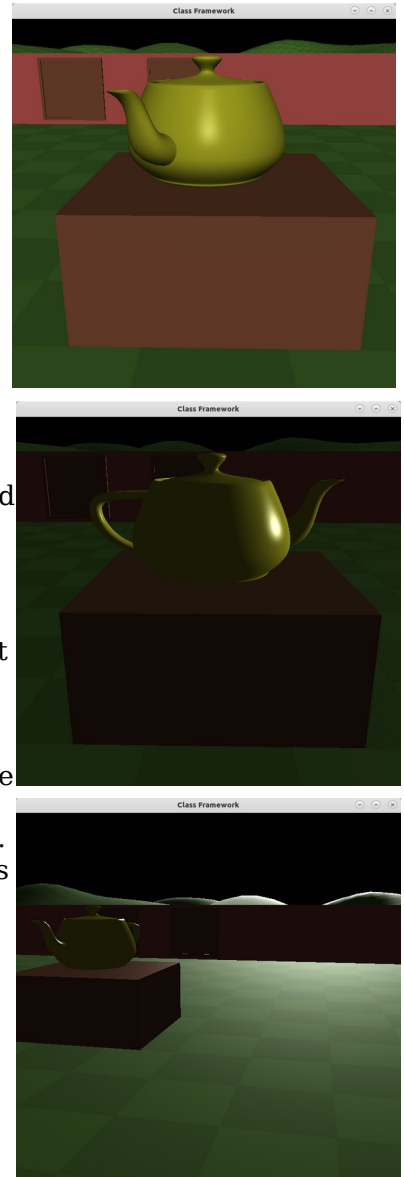
Implement a micro-facet BRDF lighting calculation in a GLSL shader program, replacing the very cheap lighting provided by the original framework.

## Instructions

This project will involve making changes to three separate programming levels in the framework: the CPU application, and the GLSL vertex and pixel shaders. All three levels already exist in the framework, although you may have been unaware of the two shaders.

The detail of passing lighting values from the application to the shaders in preparation for the calculation of a pixel's color is detail in the table on the last page. Several things are noted here:

- Each object in the scene is provided with some lighting parameters (diffuse color, specular color, and shininess). These should be passed to the shader just before the object is drawn. See the various **glUniform...** calls in procedure **Object::Draw** in **object.cpp**.
- The position of the light can be controlled by SHIFT-left-mouse-button and SHIFT-mouse-scroll-wheel. The resulting position of the light is calculated in a variable named **lightPos**, and this value must be passed into the shaders. See **glUniform3fv(loc, 1, &(lightPos[0]))** in **DrawScene** in **scene.cpp**.
- The brightness of the light, and the ambient light are NOT provided in the framework. Suggested values are provided in the table. These values should be passed into the shaders.
- The normal vector for each vertex must be calculated very carefully. In particular, it involves the **inverse** of the modeling matrix. Details are provided below.
- The final lighting calculation involves three unit length vectors,  $L$ ,  $V$ , and  $N$ . These vectors must be calculated for each pixel and normalized. Details are provided below.
- Your calculations may need the following GLSL functions and operations:
  - **dot(A,B)** returns the dot product of two vec3's.
  - **normalize(V)** returns V scaled to unit length.
  - **max(a,b)** returns the maximum of two values.
  - **pow(v,a)** returns  $v^a$
  - **A+B** adds two vec3's
  - **s\*A** multiplies a vec3 by a scalar
  - **A\*B** multiplies two vec3's component-wise. This is not a normal geometric operation for vectors, but it's just what is needed for color calculations.
  - Assignment of your vec3 calculation into the vec4 **FragColor** can be accomplished using "swizzling":  
**FragColor.xyz = <some vec3 calculation>;**
  - Vectors can be created with a constructor-like notation:  
**vec3 Ambient=vec3(0.4,0.4,0.4);**



## Framework files

Most of the code in this project will be placed in the following 3 files:

1. CPU code will go mostly in the **DrawScene** procedure in the file **scene.cpp**.
2. Vertex Shader code goes in file **lighting.vert**.
3. Fragment (A.K.A Pixel) Shader code goes in **lighting.frag**.

## Surface Parameters

All of the objects in the scene are created with calls of the form

**new Object(polygons, id, Kd, Ks, alpha) // Specifying a visible object**

where **Kd**, **Ks**, and **alpha** are the parameters that control the surface color/appearance. Please ignore lines of the form

**new Object(NULL, nullId) // For grouping other objects into a hierarchy**

for now. (You may choose to deal with these in project 5.)

You are required to change some of these values, but may change any of them:

- **Kd** is the base color of an object. For example, the teapot uses (0.5, 0.5, 0.1) for a yellow color in an attempt to look vaguely brass-like.
- **Ks** is the color/brightness of the specular reflection. Most objects have purely white specular reflections (with colored metals bring the exception). Currently, these values are set reasonably well for Phong lighting, but those values are **way** too high for micro-facet BRDF lighting. Try (0.03, 0.03, 0.03) or less for most objects.
- **alpha** is the surface roughness parameter. Use 2-5 for rough surfaces (like the ground), 5-20 for moderately smooth surfaces, and 50-100 for polished shiny surfaces.

## Lighting Parameters

The brightness of the light is unspecified in the framework and so implicitly (1,1,1). This is fine for Phong lighting, but must be more like (3,3,3) for micro-facet BRDF calculations.

The position of the light is currently controlled by SHIFT-left-mouse-button and SHIFT-scroll-wheel, and the resulting position is made available to the shaders. Please leave these parts of the framework unchanged.

## What/when/how to submit

Use Moodle to submit a single zip file of your project.

- Create a zip file (of any name) containing your source code **\*.cpp**, **\*.h**, **\*.vcproj**, **\*.vert** and **\*.frag** files *and nothing else*.
- **DO NOT** include the following in your submitted ZIP file: the **libs** folder (I created that and don't need any more copies), the **.vs**, **Debug**, and **Release** folders (I have my own compiler and don't need the output of yours), various other created files like **framework.VC.db**, **framework.VC.opendb**, **framework.vcxproj.user** (similar reason).
- Log onto the Moodle at "<https://distance.digipen.edu/>", and upload your zip file using the project link on the class web page.

# Flow of data for lighting in a GLSL shader

Multiple values are created and consumed at various points in the graphics pipeline. They fall into several categories:

- Most values needed for the lighting calculation (**diffuse, specular, shininess, Light, AmbientLight, EyePos, LightPos**) come from the application program running on the CPU and are passed into **uniform** (global) variables in the shaders.
- Several values (**WorldPos and normalVec**) are calculated for each vertex in the vertex shaders, and are passed (via **out** variables) to the fragment shaders (as **in** variables).
- One value (**gl\_Position**) is created in the vertex shader as a vertex transformed to the screen. It is needed to feed the graphics pipeline (clipper → scan conversion → screen pixels).
- Several values are created as local variables in the fragment shader to calculate intermediate values. For example **float NdotL = max(dot(N,L), 0.0)**

|                                       | Application                                                                                                                                                                                                                                                                                                                     | Shaders                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pixel position                        | <pre>loc=glGetUniformLocation(program-&gt;programId, "ModelTr"); glUniformMatrix4fv(loc, 1, GL_FALSE, Pntr(objectTr));</pre>                                                                                                                                                                                                    | <b>Vertex shader:</b><br><i>// Computes world position at a pixel used for<br/> // light and eye vector calculations</i><br>uniform mat4 ModelTr;<br>vec3 worldPos;<br>worldPos=(ModelTr*vertex).xyz;<br><i>// Computes the point's projection on the screen</i><br>gl_Position=WorldProj*WorldView*ModelTr*vertex;                                                                                                                                                                  |
| Normal vector                         | <pre>glm::mat4 inv = glm::inverse(objectTr); loc = glGetUniformLocation(program-&gt;programId, NormalTr"); glUniformMatrix4fv(loc, 1, GL_FALSE, Pntr(inv));</pre>                                                                                                                                                               | <b>Vertex shader:</b><br><i>// Computes normal vector<br/> // and outputs to fragment shader</i><br>uniform mat4 NormalTr;<br>out vec3 normalVec;<br>normalVec = vertexNormal*mat3(NormalTr);<br><b>Fragment Shader:</b><br><i>// Computes unit vector N (for lighting calc)</i><br>in vec3 normalVec;<br>vec3 N = normalize(normalVec);                                                                                                                                             |
| Eye & Light vectors                   | <pre>loc = glGetUniformLocation(programId, "lightPos"); glUniform3fv(loc, 1, &amp;(lightPos[0]));</pre>                                                                                                                                                                                                                         | <b>Vertex Shader:</b><br><i>// Computes vectors toward light and eye<br/> // and outputs them to the fragment shader</i><br>uniform vec3 lightPos;<br>out vec3 lightVec, eyeVec;<br>vec3 eyePos = WorldInverse*vec4(0,0,0,1)).xyz;<br>lightVec=lightPos-worldPos;<br>eyeVec =eyePos-worldPos;<br><b>Fragment Shader:</b><br><i>// Computes unit vectors L and V (for lighting calc)</i><br>in vec3 lightVec, eyeVec;<br>vec3 L = normalize(lightVec);<br>vec3 V = normalize(eyeVec); |
| Material properties                   | <pre>loc = glGetUniformLocation(program-&gt;programId, "diffuse"); glUniform3fv(loc, 1, &amp;diffuseColor[0]);  loc = glGetUniformLocation(program-&gt;programId, "specular"); glUniform3fv(loc, 1, &amp;specularColor[0]);  loc = glGetUniformLocation(program-&gt;programId, "shininess"); glUniform1f(loc, shininess);</pre> | <b>Fragment Shader:</b><br><i>// Values describing the surface</i><br>uniform vec3 diffuse; // Kd<br>uniform vec3 specular; // Ks<br>uniform float shininess; // alpha exponent                                                                                                                                                                                                                                                                                                      |
| Light & Ambient color/<br>bright-ness | <pre>vec3 Light(3,3,3), Ambient(0.4,0.4,0.4);  loc = glGetUniformLocation(program, "Light"); glUniform3fv(loc, 1, &amp;(Light[0]));  loc = glGetUniformLocation(program, "Ambient"); glUniform3fv(loc, 1, &amp;( Ambient[0]));</pre>                                                                                            | <b>Fragment Shader:</b><br><i>// Values describing the scene's light</i><br>uniform vec3 Light; // Li<br>uniform vec3 Ambient; // Ia                                                                                                                                                                                                                                                                                                                                                 |
| Final color calculation               |                                                                                                                                                                                                                                                                                                                                 | <b>Fragment Shader:</b><br><i>// The lighting calculation ...</i><br>vec3 H = normalize(L+V);<br>float LN = max(dot(L,N),0.0);<br>float HN = max(dot(H,N),0.0);<br>FragColor.xyz = ...                                                                                                                                                                                                                                                                                               |

## Grading Rubric for Project 2

### Previous projects:

|                                     |    |
|-------------------------------------|----|
| Project 1 functionality still works | 5% |
|-------------------------------------|----|

|                                         |    |
|-----------------------------------------|----|
| Functionality to move light still works | 5% |
|-----------------------------------------|----|

### Lighting parameters:

|                                             |     |
|---------------------------------------------|-----|
| Light brightness increased to about (3,3,3) | 10% |
|---------------------------------------------|-----|

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| Surface specular reflections adjusted to about (0.03, 0.03, 0.03) | 10% |
|-------------------------------------------------------------------|-----|

|                                                             |     |
|-------------------------------------------------------------|-----|
| All surface shininess values adjusted to something sensible | 10% |
|-------------------------------------------------------------|-----|

### Ambient light

|                                              |     |
|----------------------------------------------|-----|
| Implemented and set to something reasonable. | 10% |
|----------------------------------------------|-----|

### Diffuse reflection

|             |     |
|-------------|-----|
| Implemented | 10% |
|-------------|-----|

|                                           |     |
|-------------------------------------------|-----|
| Diffuse reflection tracks light correctly | 10% |
|-------------------------------------------|-----|

### Specular reflection

|             |     |
|-------------|-----|
| Implemented | 10% |
|-------------|-----|

|                                            |     |
|--------------------------------------------|-----|
| Specular reflection tracks light correctly | 10% |
|--------------------------------------------|-----|

|                            |     |
|----------------------------|-----|
| Demonstrates Fresnel flare | 10% |
|----------------------------|-----|

# Lighting

## Input parameters

Geometry (unit length vectors):

$N$  : Normal vector, vector orthogonal to the surfaces

$V$  : View vector, vector toward eye

$L$  : Light vector, vector toward lighting

Light values:

$I_a$  : Ambient light (R,G,B)

$I_i$  : Light's color/brightness (R,G,B)

Surface properties:

$K_d$  : Diffuse color (R,G,B) fractions

$K_s$  : Specular color (R,G,B) fractions

$\alpha$  : Shininess

## Phong Lighting

You may wish to start with Phong lighting, as a debugging aid, because the equation is shorter, but ultimately, this project requires a micro-facet BRDF lighting calculation.

$$I = I_a K_d + I_i K_d (N \cdot L)_+ + I_i K_s (N \cdot H)_+^\alpha$$

Remember  $N$ ,  $L$ , and  $V$  must be unit length vectors, as should be the halfway vector  $H = \frac{L+V}{|L+V|}$ .

**Note: All dot products should be clamped to non-negative values!**

Suggested starting values for various parameters:

$I_a$  : RGB values 0.1 to 0.2

$I_i$  : RGB values 0.7 to 1.3

$K_d$  : RGB values centered around 0.7 according to desired color

$K_s$  : RGB values 0.3 to 0.9

$\alpha$  : 5 to 200 for wide to narrow specular reflection respectively

## Micro-facet BRDF lighting

The general micro-facet lighting equation is

$$I = I_a K_d + I_i (N \cdot L)_+ BRDF$$

where the BRDF portion is:

$$BRDF = \frac{K_d}{\pi} + \frac{F(L, H) G(L, V, H) D(H)}{4 (L \cdot N) (V \cdot N)}$$

The F, G, and D terms exist in many variants, but here is a **nice starter set**:

- The Schlick approximation to the Fresnel term  $F$  is

$$F(L, H) = K_s + ((1, 1, 1) - K_s)(1 - L \cdot H)^5$$

- The masking term  $G$  and part of the denominator are lumped together and approximated with

$$\frac{G(L, V, H)}{(L \cdot N) (V \cdot N)} = \frac{1}{(L \cdot H)^2}$$

- The micro-facet normal distribution term  $D$  is motivated by Phong

$$D(H) = \frac{\alpha + 2}{2\pi} (N \cdot H)^\alpha$$

but has a normalizing factor so it's integral is one.

**Note: All dot products should be clamped to non-negative values!**

Suggested starting values for various parameters are similar to Phong but not identical:

$I_a$ : RGB values 0.1 to 0.2

$I$ : RGB values 3.0 or higher

$K_d$ : RGB values centered around 0.7 according to the desired color

$K_s$ : RGB values of 0.01 to 0.05

$\alpha$ : 5 to 200 for wide to narrow specular reflection respectively